Imam Abdulrahman Bin Faisal University
College of Computer Science & Information Technology
Department of Computer Science

CS 411 – Software Engineering
Term 1 – 2024/2025
December 3 – December 11

Software Test Plan (STP)

Software Quality Assurance Plan

For

Version [01]

Students:

Supervised By:
Dr. Enas Elsharaw

Copyright © 2024 by CS Year 4.

Revision History

Name	Date	Reason For Changes	Version
All members	Dec. 4, 2024	Initial version	0.1
All members	Dec. 5, 2024	Updated section 2	0.2
All members	Dec. 6, 2024	Updated section 2 and 3	0.3
All members	Dec. 7, 2024	Updated section 4	0.4
All members	Dec. 8, 2024	Updated section 5	0.5
All members	Dec. 9, 2024	Updated section 6	0.6
All members	Dec. 10, 2024	Spelling and grammar mistake	0.7
All members	Dec. 11, 2024	Complete review - Final version	1.0

Table 1 Revision History

Copyright © 2024 by CS Year 4.

TABLE OF CONTENTS

1. INTRODUCTION 1
2. TEST ITEMS 3
3. FEATURES TO BE TESTED 4
4. FEATURES NOT TO BE TESTED 4
5. APPROACH 4
6. PASS / FAIL CRITERIA 6
7. TESTING PROCESS 6
8. ENVIRONMENTAL REQUIREMENTS 7
9. CHANGE MANAGEMENT PROCEDURES 8
10. PLAN APPROVALS 8

Copyright © 2024 by CS Year 4.

TABLE OF TABLES

Table 1 Revision History 2
Table 2 Definitions and Acronym 3
Table 3 FEATURES NOT TO BE TESTED 6
Table 4 Login Test 7
Table 5 Sign In Test 8
Table 6 Password Recovery Test 8
Table 7 Create a Study Plan Test 9
Table 8 Whiteboard Interface Test 9
Table 9 Import Study Plans Test 9
Table 10 Admin Integration Testing 10
Table 11 Student Integration Testing 11
Table 12 Interface Testing 13
Table 13 Resources 24
Table 14 Schedule 25
Table 15 Risks and Assumptions 28
Table 16 PLAN APPROVALS 29

Copyright © 2024 by CS Year 4.

TABLE OF FIGURES

Figure 1 Data Validation Testing.....11
Figure 2Recovery life cycle.....15
Figure 3 Progressive VS Corrective Regression Testing.....18
Figure 4 Progressive Regression Testing Flowchart.....19
Figure 5 Corrective Regression Testing Flowchart.....20

Copyright © 2024 by CS Year 4.

1. INTRODUCTION

To guarantee that the EDUassist fulfills user expectations and satisfies their needs, it is crucial to subject every platform or application to a thorough testing process. A Software Test Plan (STP) is a vital part of this process as it provides a structured framework for examining all aspects of the product. This document outlines the testing methodology for the EDUassist, including objectives, strategies, scope, definitions, acronyms, and references.

1.1 Objectives

The purpose of the Software Test Plan (STP) is to clearly outline the procedures for conducting various types of testing as outlined in the scope. Additionally, it aims to establish criteria for suspending, resuming, and approving tests, ensuring clarity in the pass/fail standards. The document also identifies necessary resources, including hardware, software, networks, tools, and personnel. Moreover, it details the Environmental Requirements, Change Management Procedures, and a schedule specifying the duration of each testing task. Approvals from all stakeholders are also documented within the plan.

1.2 Testing Strategy

The Software Test Plan (STP) serves as a detailed guide for organizing and documenting the testing activities of the EDUassist:

- Section 2: Outlines Test Items to assess compliance with graphical user interface (GUI) requirements.
- Sections 3 and 4: Detail the functional aspects to be included in testing and those to be excluded.
- Section 5: Provides an in-depth explanation of the specific components subjected to testing.
- Section 6: Specifies Pass/Fail Criteria by defining conditions for suspension, resumption, and approval.
- Section 7: Discusses the Testing Process, covering deliverables, assigned tasks, roles, resources, and timelines.
- Section 8: Lists hardware, software, tools, security measures, publications, risks, and assumptions as part of the Environmental Requirements.
- Section 9: Explains the procedures for managing changes to the test plan.
- Section 10: Concludes with the approval process from all participants.

Copyright © 2024 by CS Year 4.

1.3 Scope

The Software Test Plan will encompass all stages of the testing process. Various types of tests will be outlined based on the specific elements and objectives involved. These include component testing, integration testing, conversion testing, job stream testing, interface testing, security testing, recovery testing, performance testing, regression testing, acceptance testing, and beta testing. It is important to note that not all features will be included in the testing scope.

1.4 Reference Material

[1] IEEE Standard for Software and System Test Documentation, IEEE Std 829-2008, Institute of Electrical and Electronics Engineers, 2008. [Online].

[2] ISO/IEC/IEEE International Standard - Software and Systems Engineering -- Software Testing -- Part 3: Test Documentation, ISO/IEC/IEEE 29119-3:2013, International Organization for Standardization, 2013. [Online].

[3] ISO/IEC 25010:2011 Systems and Software Engineering -- Systems and Software Quality Requirements and Evaluation (SQuaRE) -- System and Software Quality Models, International Organization for Standardization, 2011. [Online]. Available:

[4] ISO/IEC/IEEE International Standard - Systems and Software Engineering -- Vocabulary,ISO/IEC/IEEE 24765:2010(E), International Organization for Standardization, 2010. [Online].

[5] IEEE Standard for Software Reviews and Audits, IEEE Std 1028-2008, Institute of Electrical and Electronics Engineers, 2008. [Online].

[6] IEEE Standard for Software Quality Assurance Processes, IEEE Std 730-2014, Institute of Electrical and Electronics Engineers, 2014. [Online].

[7] ISO/IEC/IEEE 29119-1:2013 – Software and Systems Engineering – Software Testing – Part 1: Concepts and Definitions, International Organization for Standardization, 2013. [Online].

[8] ISO/IEC/IEEE 29119-2:2013 – Software and Systems Engineering – Software Testing – Part 2: Test Processes, International Organization for Standardization, 2013. [Online].

[9] ISO/IEC/IEEE 29119-4:2015 – Software and Systems Engineering – Software Testing – Part 4: Test Techniques, International Organization for Standardization, 2015. [Online].

[10] ISO/IEC 25010:2011 – Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) – System and Software Quality Models, International Organization for Standardization, 2011. [Online].

[11] Ian sommerville, Software Engineering. Bosten: Person Education,2011.

Copyright © 2024 by CS Year 4.

1.5 Definitions and Acronyms

Acronyms	Stand For	Definition
EDUassist	Education Assistance Software	A platform designed to assist students in managing studies and collaborating using AI tools.
STP	Software Test Plan	A formal document outlining the strategy, scope, and schedule for testing a software product.
GUI	Graphical User Interface	A type of user interface that allows users to interact with the software through visual elements.
QA	Quality Assurance	A systematic process to ensure the quality of software products through planned and systematic testing.
SRS	Software Requirements Specification	A document that describes the features and functionalities of the software to be developed.
SDS	Software Design Specification	A document that details the design and architecture of the software system.
UAT	User Acceptance Testing	A phase of testing where the end users validate the functionality of the software before deployment.
QA teams	Quality Assurance teams	Groups responsible for ensuring software quality by designing and executing tests, identifying defects, and verifying that products meet requirements before release. They play a crucial role in delivering reliable and user-friendly software.
API	Application Programming Interface	A set of rules and protocols for building and interacting with software applications.
LMS	Learning Management System	A software application for the administration, documentation, tracking, reporting, and delivery of educational courses.

Table 2 Definitions and Acronym

Copyright © 2024 by CS Year 4.

2. TEST ITEMS

This section will verify that all elements in EDUassist, including data, databases, and interfaces, meet the required functionality and usability standards. The SRS document provides details about the system requirements, while the SDS document contains the design and database specifications.

2.1 Program Modules

Every test is carried out by the developer and integrated into EDUassist. The system is divided into modules, such as login/authentication, study plan management, task tracking, AI-assisted whiteboard, and resource recommendations. Each module is independently testable, ensuring it meets the specified requirements. Once individual modules pass unit testing, they will be integrated and tested for seamless functionality. This report's **Section 5** will detail the testing procedures for all program modules.

2.2 Job Control Procedures

Each EDUassist component will be tested independently before proceeding with sequential testing. Job control processes, such as task scheduling, API calls, and system notifications, will be validated to ensure they are executed in the correct sequence. Functional, interface, security, and performance tests will be applied to identify and resolve any errors. Finally, acceptance and regression tests will confirm the system meets all specified requirements.

2.3 User Procedures

Every user procedure implemented in EDUassist, such as creating study plans, tracking tasks, and collaborating via the whiteboard, will be tested to ensure smooth operation. Testing will validate:

- Correctness and completeness of workflows.
- Accuracy of user documentation, ensuring users can follow step-by-step instructions effectively. This report's **Section 5** will provide further details on testing user procedures.

Copyright © 2024 by CS Year 4.

2.4 Operator Procedures

The EDUassist platform will undergo extensive testing to confirm it can be operated effectively in a production environment. These tests will verify:

- System monitoring tools for tracking uptime, user activity, and performance.
- Help desk procedures for resolving common issues like login failures and task errors.
- Backup and recovery processes to ensure system integrity during failures.

Testing will include:

- Component Testing
- Integration Testing
- Interface Testing
- Security Testing
- Performance Testing
- Regression Testing
- Acceptance Testing
- Beta Testing

The details for these procedures are explained in Section 5 of this report.

Copyright © 2024 by CS Year 4.

3. FEATURES TO BE TESTED

Admin Features:

- User Management (Create, modify, delete user accounts, reset passwords).
- System Monitoring (Generate reports, monitor user activities).
- Configuration Settings (Integrations with tools like Google Calendar).

Student Features:

- Personalized Study Plans.
- AI-Assisted Whiteboard (Collaboration and annotation in real-time).
- Task Tracking (Add, edit, complete tasks, receive reminders).
- Resource Recommendations (AI-driven suggestions for study materials).
- Video Generation (Generate short videos from whiteboard sessions).
- External Integrations (Sync with Google Calendar).

System Features:

- Authentication (Login, password recovery, two-factor authentication).
- Data Management (Store user profiles, study plans, and task progress).
- Integration Services (LMS and calendar).
- Performance Monitoring (Track user interactions, generate usage insights).

4. FEATURES NOT TO BE TESTED

All required features included a testing method to increase the dependability of our application; features or interface did not require any testing then it was not included.

Feature	Reason
Email Notification Logs	Detailed testing of email notification logs is excluded since the feature does not directly affect platform usability or critical workflows.
Non-Core Integrations	Features related to third-party integrations, such as Google Calendar features or Learning Management System (LMS)
User Interface Themes	Alternative themes for the user interface will not be tested as the current testing scope only includes the default theme.

Table 3 FEATURES NOT TO BE TESTED

Copyright © 2024 by CS Year 4.

5. APPROACH

Testing is the systematic process of examining software components to identify discrepancies between the expected and actual conditions, as well as to evaluate their features. In this section, we will explore the various testing components for the EDUassist in detail, which include:

- Component Testing
- Integration Testing
- Conversion Testing
- Job Stream Testing
- Interface Testing
- Security Testing
- Recovery Testing
- Performance Testing
- Regression Testing
- Acceptance Testing
- Beta Testing

5.1 Component Testing

Component testing is a crucial phase in software development that involves the independent evaluation of each component to ensure it meets predefined expectations. This method facilitates early identification of defects, streamlining problem-solving. Each component is examined in isolation, allowing for a focused assessment of its functionality. The completion criteria require that each component produces the intended outcomes before integration into the larger system, thereby enhancing overall reliability.

- **Login Test**

Test ID	LoginTest
Prerequisite	Having an account
Test Procedure	1. Enter correct username and password. 2. Enter correct username and wrong password. 3. Enter wrong username and correct password. 4. Leave one or both fields empty.
Expected Result	1. Successful login and move on to homepage. 2. An error message "Wrong username/password" is displayed. 3. An error message "Wrong username/password" is displayed. 4. An error message "Please fill in the fields" is displayed.
Actual Result	As the expected
Verified (Yes/No)	Yes

Table 4 Login Test

Copyright © 2024 by CS Year 4.

- **Sign In Test**

Test ID	SignInTest
Prerequisite	Having an account
Test Procedure	1. Click on "Sign In" Button. 2. Enter valid credentials. 3. Enter invalid credentials.
Expected Result	1. Redirected to the sign-in page. 2. Successful redirection to the user dashboard. 3. Display error message: "Invalid credentials."
Actual Result	As the expected
Verified (Yes/No)	Yes

Table 5 Sign In Test

- **Password Recovery Test**

Test ID	PasswordRecoveryTest
Prerequisite	User must have an account
Test Procedure	1. Click on "Forgot Password" Button. 2. Enter registered email address. 3. Click on the link in the recovery email. 4. Enter new password and confirm.
Expected Result	1. Redirected to password recovery page. 2. Confirmation message: "Recovery email sent." 3. Redirected to password reset page. 4. Confirmation message: "Password reset successful."
Actual Result	As the expected
Verified (Yes/No)	Yes

Table 6 Password Recovery Test

- **Create a Study Plan Test**

Test ID	CreateStudyPlanTest
Prerequisite	User logged in
Test Procedure	1. Upload a syllabus file in PDF or Word format. 2. Confirm the upload and initiate AI study plan generation. 3. Review the generated study plan.
Expected Result	1. File uploads successfully without errors. 2. AI generates a study plan based on the syllabus content. 3. Study plan meets user expectations and requirements.
Actual Result	As the expected
Verified (Yes/No)	Yes

Copyright © 2024 by CS Year 4.

Table 7 Create a Study Plan Test

- **Whiteboard Interface Test**

Test ID	WhiteboardTest
Prerequisite	User logged in
Test Procedure	1. Access the whiteboard interface. 2. Use drawing tools on the whiteboard. 3. Save the whiteboard changes.
Expected Result	1. Whiteboard interface is displayed. 2. Drawings are displayed accurately. 3. Confirmation message: "Whiteboard saved."
Actual Result	As the expected
Verified (Yes/No)	Yes

Table 8 Whiteboard Interface Test

- **Import Study Plans Test**

Test ID	WhiteboardTest
Prerequisite	User logged in
Test Procedure	1. Navigate to "Import Study Plans" section. 2. Upload a valid study plan file. 3. Upload an invalid file format.
Expected Result	1. Import interface is displayed. 2. Confirmation message: "Study Plan imported successfully." 3. Error message: "Invalid file format."
Actual Result	As the expected
Verified (Yes/No)	Yes

Table 9 Import Study Plans Test

Copyright © 2024 by CS Year 4.

5.2 Integration Testing

Integration testing ensures that EDUassist modules function cohesively by testing interactions and data flow between components. Separate test cases are designed for Admin and Student modules to validate specific functionalities.

5.2.1 Admin Integration Testing

Table 10 shows test cases of the integration among components beginning with the Admin homepage.

Test ID	Admin_Integration_001
Prerequisite	Admin has successfully logged in.
Test Procedure	Access system monitoring, generate activity reports, create or modify user accounts, and manage configuration settings for external integrations like Google Calendar.
Expected Result	- Admin can navigate seamlessly between user management and system monitoring features. - Activity logs are accurately displayed and updated in real time. - User accounts are created, modified, or deleted without errors. - Integration settings with Google Calendar are updated and reflected immediately
Actual Result	As the expected
Verified (Yes/No)	Yes

Table 10 Admin Integration Testing

5.2.2 Student Integration Testing

Table 11 shows test cases of the integration among components beginning with the Student dashboard.

Test ID	Student_Integration_001
Prerequisite	Student has successfully logged in.
Test Procedure	Test interactions between study plan generation, task tracking, AI-driven resource recommendations, and notifications. Verify synchronization with Google Calendar for task deadlines.
Expected Result	- Study plans are generated correctly based on uploaded syllabus files. - Tasks are added, edited, or marked as complete, and updates trigger accurate notifications. - AI resource recommendations align with the student's study plan

Copyright © 2024 by CS Year 4.

	and progress. - Task deadlines synchronize seamlessly with Google Calendar.
Actual Result	As the expected
Verified (Yes/No)	Yes

Table 11 Student Integration Testing

5.3 Conversion Testing

Data plays a vital role in software applications, being integral to various functionalities. Conversion testing is crucial for ensuring that data is accurately transformed from one format to another, particularly when preparing data for use within the application rather than focusing solely on web-based formats.

Data conversion can be executed either automatically through specialized tools or manually by extracting data from the source database and inputting it into the target database.

Technique: **Data Validation Testing**

Data validation testing is designed to confirm the accuracy and integrity of data, ensuring that it is complete and devoid of errors. This process guarantees that data can be transformed as needed without any loss and accurately reflects the expected information based on the provided inputs.

In **Figure 1** shows the flow chart for Data Validation Testing in the EDUassist. It outlines the process from data input through integrity, completeness, and format checks, leading to either data approval or error handling, which notifies users of issues and suggests corrections.

Completion Criteria: Successful conversion is confirmed when data transitions from the old system format to the new system format without any alterations to the data elements or types.

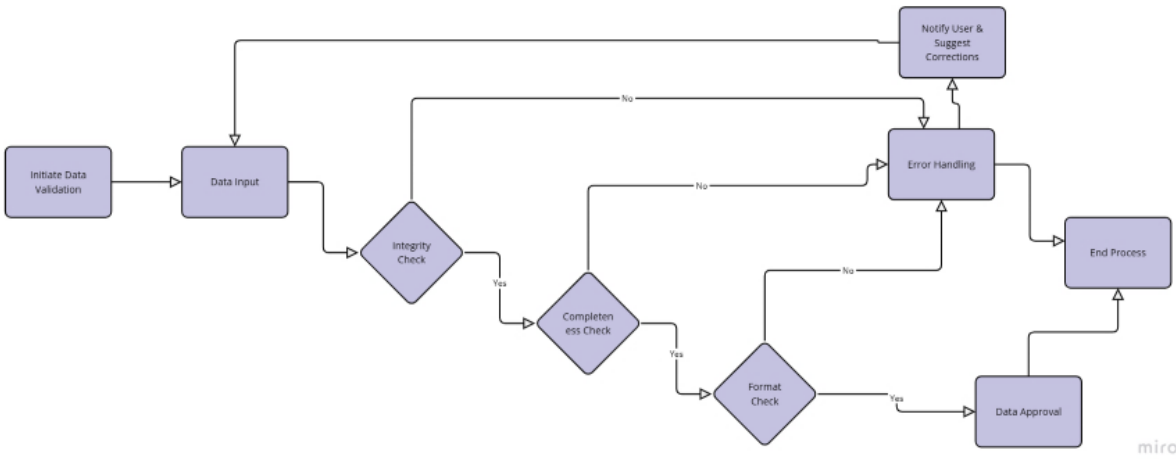


Figure 1 Data Validation Testing

Copyright © 2024 by CS Year 4.

5.4 Job Stream Testing

Job stream testing is a critical process in the EDUassist that focuses on executing a series of functions in a specific sequence, triggered by predefined actions. Each job within this stream has unique characteristics and dependencies, ensuring that the output of one job serves as the input for the next. This creates a continuous flow of data and processes essential for optimal system performance.

Technique: Test Stream of Functions

In this approach, the testing involves monitoring the execution of functions in the designated order, validating that each job interacts correctly with others within the stream.

Completion Criteria:

Job stream testing is deemed successful when the following criteria are met:

- A series of functions is executed in the correct order based on predefined actions.
- Each job maintains its distinct properties and dependencies.
- The output of one job correctly feeds into the subsequent job, establishing a seamless data flow.
- The overall operation of the system remains uninterrupted, ensuring that processes integrate smoothly and efficiently.

This thorough testing ensures that the EDUassist operates as intended, maintaining a reliable and efficient workflow.

Copyright © 2024 by CS Year 4.

5.5 Interface Testing

The objective of interface testing within the EDUassist system is to systematically identify and rectify any flaws present in the system, thereby ensuring that all interactions function as intended. This testing is essential for addressing any defects within the software. An overview of the interface testing process is detailed in **Table 12** .

Interface Name	Description and Expected Outcome
User Login Interface	Ensures that users can log in using valid credentials. If valid, the user is redirected to their dashboard.
Sign Up Interface	Allows users to register with their details. If valid, a new user account is created and a welcome page is displayed.
Password Recovery	Enables users to recover their password by providing their registered email. If valid, a reset link is sent to the user.
Dashboard Interface	Displays personalized user information. If valid, the interface shows all accessible features based on the user's role.
Study Plan Import	Allows users to upload and import study plans. If valid, the system processes the file and displays a confirmation message.
Whiteboard Interface	Enables users to use tools for collaboration. If valid, changes are saved and a confirmation message is shown.
Admin Panel	Provides administrators access to manage users and system settings. If valid, requested changes are successfully applied.
Resource Access	Users can access learning resources and AI suggestions. If valid, resources load, and relevant suggestions are displayed.
Collaboration Tools	Facilitates real-time interaction among users. If valid, updates are visible to all participants without errors.
Google Calendar Sync	Integrates Google Calendar to manage schedules. If valid, calendar events sync successfully with the platform.

Table 12 Interface Testing

Copyright © 2024 by CS Year 4.

5.6 Security Testing

Security testing is a critical aspect of the EDUassist system, aimed at ensuring that the application's control and auditability features are fully functional and effective. This type of testing verifies that the system is resilient against unauthorized access, data breaches, and other security threats, thereby safeguarding sensitive user information and maintaining the integrity of the application.

Technique: Security Control Assessment

This technique involves evaluating the various security controls implemented within the EDUassist system. Testers assess the effectiveness of authentication mechanisms, authorization processes, and data encryption methods to ensure compliance with security standards and best practices.

Completion Criteria:Security testing is deemed successful when the following conditions are met:

- All authentication mechanisms function correctly, preventing unauthorized access to the system.
- Authorization processes are validated to ensure that users can only access data and functions for which they have explicit permissions.
- Data encryption methods are effective, ensuring that sensitive information is securely stored and transmitted.
- Audit trails are accurately maintained, enabling comprehensive tracking of user actions and system changes for accountability.

By conducting thorough security testing, the EDUassist system can ensure that its security measures are robust and effective, providing users with a safe and reliable platform. This rigorous approach to security testing is essential for identifying vulnerabilities and mitigating risks, ultimately contributing to the overall trustworthiness and resilience of the application.

Copyright © 2024 by CS Year 4.

5.7 Recovery Testing

Recovery testing is a critical aspect of non-functional testing in software engineering, aimed at evaluating a system's capability to recover from failures. This methodology involves deliberately inducing faults within the system and subsequently confirming the effectiveness of the recovery mechanisms. The primary objective is to ensure that the system can revert to a predetermined state of integrity and successfully reprocess transactions up to the failure point.



Figure 2Recovery life cycle

The recovery process can be systematically categorized into several essential stages:

1. **Standard Operation**: The system, composed of hardware, software, and firmware, operates harmoniously to achieve its intended objectives without interruption for a designated period.
2. **Occurrence of Disasters and Failures:** Disruptions to the system may arise from software anomalies, such as invalid inputs or crashes, hardware malfunctions, or external incidents like fires or natural disasters that compromise data integrity.
3. **Interruption Phase:** This phase is often the most challenging, resulting in potential business losses, strained relationships, and missed opportunities. Organizations should establish comprehensive backup strategies to minimize the impact of such disruptions, even if complete prevention is not feasible.
4. **Disaster Recovery Implementation:** When a robust backup plan and effective risk mitigation strategies are in place, recovery can be executed with minimal time and resource expenditure. Recovery testing can be performed efficiently, enabling the designated team to restore the system promptly.
5. **Restoration of Processes and Data:** This stage involves the reconstruction of all processes and data to reinstate normal operations. Restoration encompasses rebuilding

Copyright © 2024 by CS Year 4.

directories, re-establishing installation files, and configuring settings, while ensuring proper documentation is maintained for accurate recovery.

Completion Criteria: After the restoration of files and directories, the following validation checks should be conducted to confirm successful recovery:

- Verify the accessibility of files by opening them.
- Rename any corrupted document folders appropriately.
- Compare the number of files in the restored directories with those in the original directories.
- Ensure the integrity of file and directory structures.
- Confirm that the data can be effectively browsed or modified.

5.8 Performance Testing

Performance testing is essential for ensuring that the EDUassist platform meets customer expectations regarding response time, availability, portability, and scalability. The primary goal is to minimize response time, measuring the speed at which the platform operates. Users should not experience long wait times, especially under varying load conditions. Additionally, the platform must be consistently available for access by users, ensuring uninterrupted service.

The EDUassist platform should also demonstrate portability, functioning effectively across various operating systems, including Android, Windows, and others. Furthermore, it must be scalable to accommodate anticipated growth and increased user demands in the future.

Some performance tests to be implemented include:

- **Maximum Transactions Per Second**: Assessing the highest number of transactions that can be processed in one second.
- **User Load Capacity**: Determining the maximum number of simultaneous users the platform can handle before experiencing crashes.
- **Cross-Platform Functionality**: Verifying that the platform operates as expected across different operating systems.
- **Speed Under Varying Internet Conditions**: Evaluating the platform's performance under different internet connection speeds.
- **Database Backup Verification**: Ensuring that the platform successfully backs up the database daily.
- **File Upload Speed**: Measuring the speed at which users can upload pictures and other files.

Technique: Testing is conducted to ensure that the application performs to customer expectations regarding response time, availability, portability, and scalability.

Completion Criteria:

Copyright © 2024 by CS Year 4.

- Acceptable response time under load conditions.
- Continuous availability when accessed by users.
- Compatibility across all major operating systems and devices.
- Scalability to support any number of users without performance degradation.

5.9 Regression Testing

Regression testing is a crucial phase in the software testing process, performed after any code modification to ensure that these changes do not introduce new bugs into the EDUassist. Updates to the code can potentially lead to defects that affect previously stable functionalities. To manage this risk effectively, QA teams maintain a suite of regression test cases focused on key features, which are re-executed each time code changes are made. This strategy helps save time and maximize testing efficiency.

Modifications to the software can vary in scale. Minor changes might include simple edits, such as adding or deleting a few lines of code, while major modifications could involve altering or replacing entire modules or subsystems. During development cycles, the focus often shifts to testing new functionalities, which can lead to overlooking the stability of existing features. However, end-users frequently rely on these established functionalities, making it imperative that any new features integrate seamlessly without disrupting the existing system.

Regression testing serves to verify that the previously tested functionalities continue to operate correctly after changes are implemented. By identifying and addressing defects early in the software development lifecycle, regression testing enhances the overall user experience. It also ensures business continuity amid rapid software changes. Additionally, the use of automated test cases reduces execution time, allowing the testing team to cover more areas of the software more efficiently.

Completion Criteria: Regression testing is deemed complete when all previously passed test cases continue to pass, confirming that no new defects have been introduced and that existing functionalities remain intact.

Technique: Based on the nature of the modifications, two main types of regression testing can be employed:

- **Progressive Regression Testing:** This approach uses a modified specification to incorporate new feature requirements. It generates more significant results when specific changes are made, as new test cases are developed alongside the modifications. This testing ensures that elements from previous versions are not compromised in the updated version.
- **Corrective Regression Testing:** In this approach, the specification remains unchanged. This is significant because most test cases from the previous testing plan are still valid, accurately reflecting the input-output relationships. Existing test cases can be reused effectively for current testing needs.

Copyright © 2024 by CS Year 4.

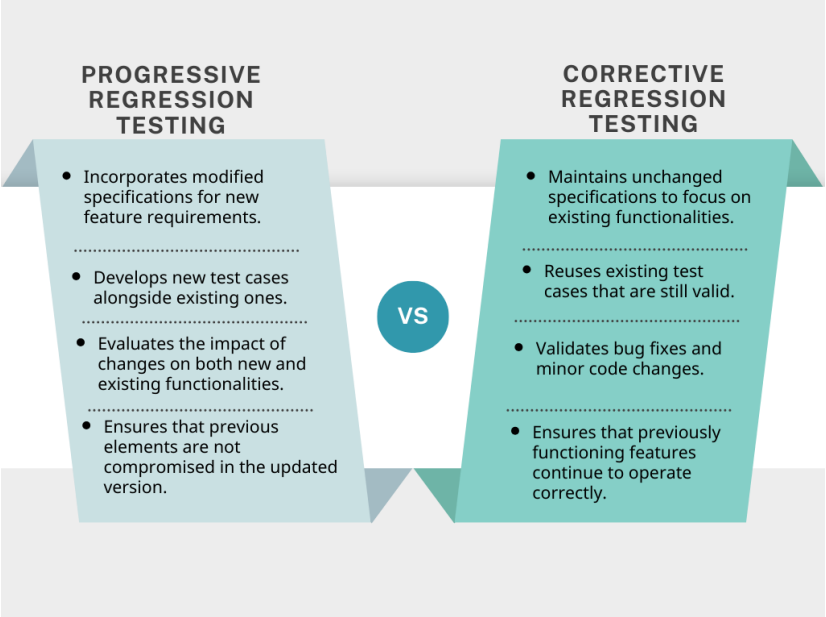


Figure 3 Progressive VS Corrective Regression Testing

Copyright © 2024 by CS Year 4.

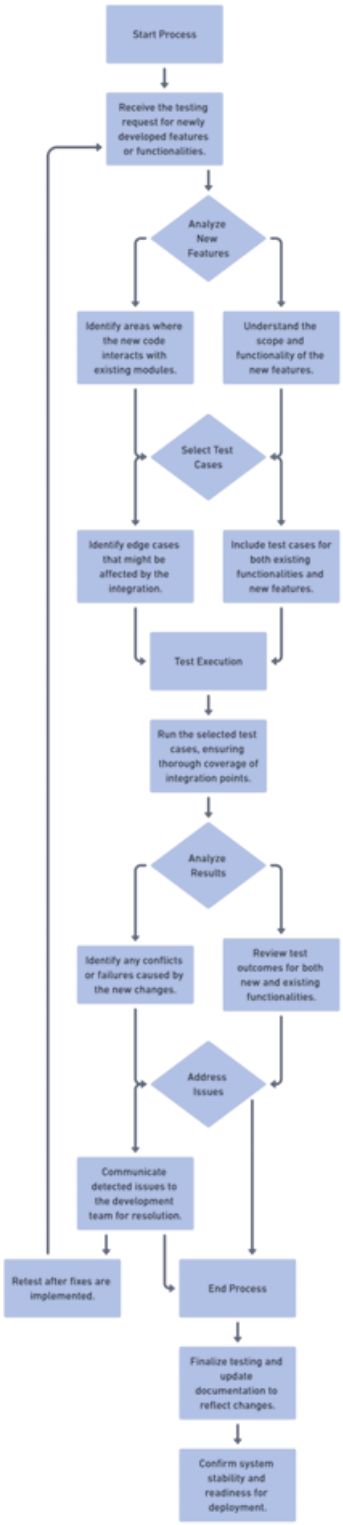


Figure 4 Progressive Regression Testing Flowchart

Copyright © 2024 by CS Year 4.

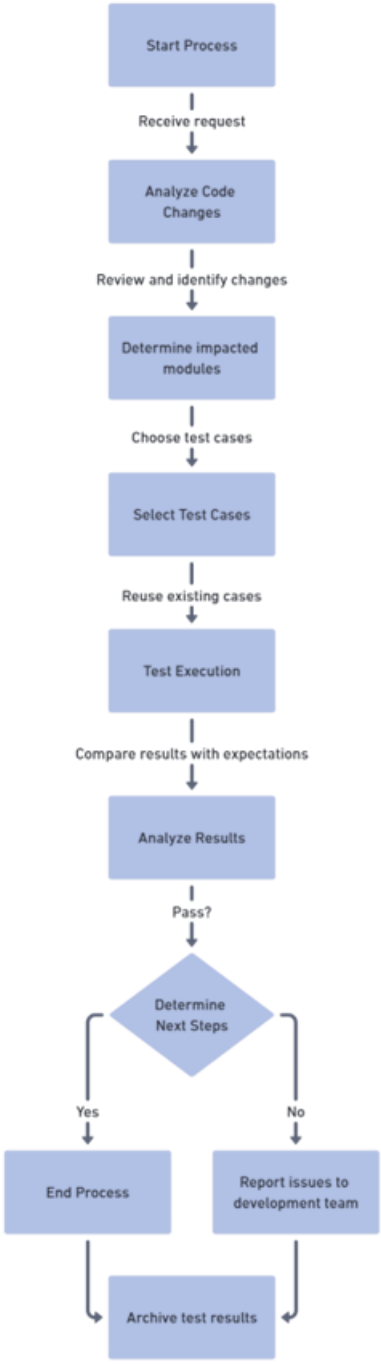


Figure 5 Corrective Regression Testing Flowchart

Copyright © 2024 by CS Year 4.

5.10 Acceptance Testing

Acceptance testing for the EDUassist is performed at the end of the software testing process to verify that the platform meets the specified requirements set forth by stakeholders before delivery to the client. This phase is critical, as collecting feedback from stakeholders enables the identification and resolution of usability issues, thereby optimizing time and cost efficiencies prior to the official release of the software.

Technique: The process involves gathering detailed feedback to ensure the software operates effectively and aligns with user expectations.
Completion Criteria: The application will only be released if it is verified to be free of errors.

5.11 Beta Testing

Beta testing for EDUassist involves making the system available to a select group of users to gather comprehensive feedback on various functionalities. EDUassist collaborates with a specialized testing company to conduct thorough assessments. Based on the feedback from these testing experts, the development team will evaluate suggested changes, identify any errors, and implement necessary modifications. The goal is to enhance the system's performance, ensuring a smooth and efficient user experience.

Technique: This process involves deploying the system to a selected group of users to collect their feedback on usability and functionality, facilitating real-world evaluation and iterative improvements.

Completion Criteria: The entire system must undergo extensive testing to confirm that it meets all functional requirements and performs as intended.

Copyright © 2024 by CS Year 4.

6. PASS / FAIL CRITERIA

For EDUassist, the following criteria must be applied to each component in the system to determine whether it passes or fails:

- The specifications detailed in the Software Requirements Specification (SRS) must be met for all user functionalities.
- The system must adequately handle any incorrect input from users.
- All test cases need to be completed successfully without any critical defects.

6.1 Suspension Criteria

Testing will be paused under the following conditions:

- **Resource Unavailability**: Key testing personnel or essential tools become unavailable.
- **Critical Issues:** Major problems arise that significantly impede progress, such as system crashes or data loss.
- **Environmental Maintenance:** Scheduled updates or maintenance are required for the testing environment to ensure stability.
- **Critical Defects**: If any critical defects are identified during testing that prevent the system from functioning (e.g., failure to log in, inability to save study plans), further development and testing will be halted until these issues are resolved.
- **Security Vulnerabilities**: Discovery of significant security vulnerabilities that compromise user data or system integrity will result in immediate suspension of all activities until a resolution is implemented.

6.2 Resumption Criteria

Testing may be resumed when the following conditions are met:

- **Resource Availability**: All necessary testing resources and personnel are available and operational.
- **Issue Resolution**: All critical issues that led to the suspension have been resolved satisfactorily.
- **Environment Stability**: The testing environment is confirmed to be stable and functioning as intended.
- **Security Remediation:** Any security vulnerabilities that led to suspension have been addressed, and the system has passed a security assessment.
- **Compliance Confirmation**: The system has been reviewed and confirmed to be compliant with all applicable regulations, ensuring that user data is protected and handled appropriately.

6.3 Approval Criteria

Test results will be deemed acceptable if the following criteria are fulfilled:

- **Functionality Coverage:** All core functionalities of EDUassist must pass their respective tests without any significant defects.

Copyright © 2024 by CS Year 4.

- **Performance Standards:** Performance metrics meet or exceed the predefined benchmarks established for the system.
- **Security Compliance**: No critical security vulnerabilities are identified during testing.
- **Regulatory Adherence:** The software complies with all relevant standards and regulations applicable to educational software.
- **User Acceptance Testing (UAT):** At least 90% of end users must successfully complete critical tasks such as user registration, study plan creation, and resource access during UAT, with positive feedback on usability and functionality.

This set of criteria ensures that the testing process for EDUassist is comprehensive, allowing only EDUassist that meets the required standards to be approved for deployment.

7. TESTING PROCESS

This section outlines the methods, criteria, deliverables, tasks, responsibilities, resources, and schedule associated with testing activities for the EDUassist platform.

7.1 Test Deliverables

The sole deliverable for the testing process is the **Software Test Plan (STP)** document itself. This document encompasses all testing strategies, methods, criteria, and related information, serving as the comprehensive record for the EDUassist testing activities.

7.2 Testing Tasks

Listed below are the testing tasks:

- Define test objectives and scope.
- Identify test items and required resources.
- Develop test cases, scripts, and scenarios.
- Configure the test environment.
- Perform unit, integration, system, and acceptance testing.
- Execute automated and manual test cases.
- Record test results and incidents.
- Review test results for compliance with requirements.
- Document findings in summary and incident reports.
- Validate fixes for identified issues.
- Confirm completion of all tests or document unresolved issues.
- Deliver the finalized Software Test Plan (STP).

Copyright © 2024 by CS Year 4.

7.3 Responsibilities

During the testing process, the team members are responsible for following the specifications of every component and how it works with the system. This includes handling errors that may occur during the testing process.

7.4 Resources

Resource Type	Details	Purpose
Human Resources	- **Testers**: QA engineers to design and execute test cases. - **Developers**: To fix defects identified during testing. - **Technical Support**: To assist with environment setup and issue resolution.	To ensure comprehensive test coverage and timely resolution of issues.
Hardware	- **Personal Computers**: Laptops/desktops - **Mobile Devices**: Android, iOS . - **Servers**: Cloud servers for hosting and testing backend services.	To test EDUassist across multiple platforms and devices.
Software Tools	- **Selenium**: For automated functional testing. - **BrowserStack**: For cross-browser and cross-platform testing. - **JIRA** and **TestRail**: For test case and issue management.	To facilitate test automation, management, and tracking.
Documentation	- **SRS Document**: Provides system requirements. - **SDS Document**: Defines system design. - **Testing Framework Guides**: Documentation for Selenium, Postman, etc.	To guide the test design and execution phases.
Network and Connectivity	- Stable high-speed internet	To ensure seamless cloud-based testing and integration with third-party services (e.g., Google Calendar).
Security Measures	- Access control to testing environments. - Encrypted data storage for test inputs and outputs.	To ensure secure handling of sensitive test data.

Table 13 Resources

Copyright © 2024 by CS Year 4.

7.5 Schedule

To get satisfying results, you must manage the project correctly. Thus, testing tasks are scheduled according to the following table.

Task	Date
Software Requirement Specification (SRS)	November 2 , 2024
Software Design Specification (SDS)	November 24, 2024
Create test cases	December 4, 2024
Execute testing activities	December 5, 2024
Troubleshoot errors occurred during testing	December 7, 2024
Modify the system accordingly	December 8, 2024
Modify Software testing plan (STP)	December 11, 2024
Test Delivery	December 12, 2024

Table 14 Schedule

Copyright © 2024 by CS Year 4.

8. ENVIRONMENTAL REQUIREMENTS

This section encompasses the environmental requirements for testing EDUassist, including hardware, software, security measures, tools, publications, risks, and assumptions. These requirements ensure the system can be tested efficiently and securely under appropriate conditions.

8.1 Hardware

To perform testing on EDUassist activities, the following hardware is required:

- **Personal Computers**:
  - Desktops and laptops with at least Intel Core i5, 16GB RAM, and SSD storage for performance testing.
- **Internet Connectivity**:
  - Stable high-speed internet (minimum 50 Mbps) to support cloud testing and synchronization.
- **Mobile Devices**:
  - Android devices (v10 and above) and iOS devices (v14 and above) for cross-platform testing.

8.2 Software

To carry out testing for EDUassist, the following software tools are required:

- **Programming and Scripting Tools**:
  - Python for test automation scripting.
- **Database Management**:
  - MySQL for testing database interactions.
- **Frameworks**:
  - Selenium for automated UI testing and Postman for API testing.
- **Version Control**:
  - GitHub for managing test cases and updates

8.3 Security

To ensure system security, the following measures are implemented:

- **Access Control**:
  - Only authorized users can access the system using email/phone and password. Two-factor authentication (2FA) is required for sensitive operations.
- **Password Recovery**:
  - Users can reset passwords through an OTP sent to their registered email or phone number.
- **Data Protection**:
  - Encryption is used for test data during storage and transmission to comply with data privacy regulations.

Copyright © 2024 by CS Year 4.

8.4 Tools

The following tools are required for testing EDUassist:

- **Automation Tools**:
  - Selenium for functional testing.
- **Validation Tools**:
  - BrowserStack for cross-browser and cross-device testing.
- **Test Management**:
  - JIRA and TestRail for managing and tracking test cases.
- **Accessibility Checkers**:
  - W3C Validator for web accessibility and compliance

8.5 Publications

The testing of EDUassist is supported by the following documents:

- Software Requirements Specification (SRS).
- Software Design Specification (SDS).

Copyright © 2024 by CS Year 4.

8.6 Risks and Assumptions

In order to ensure that EDUassist meets the specified requirements and performs effectively, it must go through multiple stages of testing. During these stages, challenges such as delays in resource availability, unforeseen technical issues, or time constraints may arise. These risks and assumptions are identified to proactively address potential obstacles and provide contingency plans to minimize their impact on testing activities. The table below outlines the significant risks, their explanations, and corresponding contingency plans.

Risks	Explanation	Contingency Plan
Test Item Availability	If the test item is not available, testing activities will halt, causing delays in the project timeline.	Utilize an alternative item that is currently accessible while waiting for the specific test item to become available.
Test Resource Availability	If required testing resources are unavailable, progress will be impeded, delaying associated test activities.	Identify and utilize alternative resources that fulfill the same purpose to continue testing without disruption.
Time Constraints	Time constraints may result in exceeding scheduled durations or unexpected delays due to unforeseen circumstances.	Create a contingency schedule (Plan B) at the outset of the project to manage unexpected changes and ensure timely completion of testing activities.

Table 15 Risks and Assumptions

9. CHANGE MANAGEMENT PROCEDURES

The Software Test Plan Change Management **Process** involves three key steps: change initiation, change review, and change authorization. The process begins with change initiation, where a need for a change is identified, such as a discovered defect, updates to requirements, or scope alterations. The proposed change is documented, detailing the rationale, impacted areas, and priority level, and a formal change request is submitted to the Change Control Board (CCB) or relevant stakeholders. In the change review phase, the CCB or stakeholders evaluate the request, assessing its potential impact on the test plan, including resource allocation, schedule adjustments, and test coverage. This review ensures the proposed change aligns with project goals and constraints. Finally, during change authorization, the CCB approves, rejects, or defers the change based on the review. The decision is documented, including the rationale and any conditions for approval, and communicated to the relevant team members. The test plan and associated documentation are then updated to reflect the approved changes. This structured process ensures changes are effectively managed, reducing disruption and maintaining project alignment.

Copyright © 2024 by CS Year 4.

10. PLAN APPROVALS

Name	Signature
	Agree
	Agree
	Agree
	Agree

Table 16 PLAN APPROVALS

Copyright © 2024 by CS Year 4.